Relatório de Estado — Caribbean Pirates GPT

Medusa + Davy-Jones

Versão: 1.0
Data: 19 de outubro de 2026
Horário da verificação: 09:10–09:40 (-03:00)
Servidor: medusa — Ubuntu Server 26.04.1 LTS, kernel 7.0.0-38-generic
Classificação: Confidencial — uso interno
Base documental: `C:\Intel\medusa\projeto-medusa_e_davy-jones_17092026\md` (15 documentos) + verificação direta no host
Relação com o modelo anterior: segue o formato de `Relatorio_001_2030_01102026.md`. Onde este relatório divergir daquele, prevalece este, porque os dados foram verificados ao vivo em 19/10/2026.

> **Nota de leitura:** os símbolos 🟢, 🟡 e 🔴 indicam, respectivamente, componente operacional e verificado, componente parcial ou em configuração, e componente ausente ou bloqueado. Eles descrevem o estado de hoje — não uma promessa de arquitetura.

Em números

Indicador	Valor verificado
Containers em execução	9 de 9
Serviços systemd ativos	4 de 4
Testes unitários	134 — 0 falhas
Corpus canônico	2 documentos / 206 chunks
Vetores no Qdrant	1.850 pontos
Backups operacionais	31 pontos / 2,9 GB
Última réplica remota	`COMPLETE`
Targets Prometheus	2 de 2 `UP`
Uptime da VM	9 dias e 10 horas
Uso de disco	12% (619 GB livres)

---

1. Situação geral

Componente	Situação
Medusa Core	🟢 Operacional
Davy-Jones (API + UI)	🟢 Operacional
Autenticação e RBAC	🟢 Operacional
Cadastro Universal (C1/C2)	🟢 Implementado
Biblioteca de documentos	🟢 Implementada
OCR documental	🟢 Operacional
Geração DOCX controlada	🟢 Operacional
Qdrant	🟢 Operacional
Corpus canônico (206 chunks)	🟢 Preservado
Workspace documental	🟢 Operacional
Ollama + RTX 5090	🟢 Operacional
Mem0	🟢 Operacional
Docling	🟢 Operacional
RAG Worker	🟢 Operacional
SearXNG	🟢 Operacional
Caddy / TLS interno	🟢 Operacional
Backup local	🟢 Operacional
Backup remoto SMB/Tailscale	🟢 Operacional
Prometheus + Grafana + node-exporter	🟢 Operacional
PostgreSQL + pgvector (infra)	🟢 Disponível
PostgreSQL/pgvector como base da aplicação	🟡 Preparado, ainda sem uso
Redis	🟡 Disponível, sem uso direto identificado
Observabilidade completa (logs, alertas, uptime)	🟡 Parcial
Métricas de aplicação e por container	🟡 Parcial
Geração jurídica controlada	🟡 Em revisão (M5.24-R1)
Diagnóstico D4 de não determinismo	🔴 Autorizado, não executado
Validação autenticada em navegador	🟡 Pendente de senha humana

Componente	Situação
Reconstrução do rótulo removido	🟡 Pendente de autorização
Open WebUI	🔴 Não instalada
Nginx / HTTPS :443	🔴 Não adotados (Caddy em 5080/3443)
Loki / Alertmanager / Uptime Kuma	🔴 Não implantados
Backup independente em nuvem	🔴 Pendente
Planner avançado do Agent	🔴 Não implementado
Geração jurídica autônoma	🔴 Deliberadamente desligada

Resumo em uma frase: o núcleo funcional está pronto e estável; o que falta está concentrado em observabilidade completa, continuidade externa, decisão sobre a camada de dados, validação final da geração jurídica e pendências pontuais de autorização — não em reconstrução do que já existe.

---

2. O que já está concluído

2.1 Linha do tempo consolidada

Período	Entrega
17–19/09/2026	Arquitetura, baseline Proxmox, VM, GPU passthrough, Docker e serviços centrais
19–25/09/2026	Consolidação do corpus, cadeia M4/M5, índice estruturado, validação Qdrant
28/09/2026	Cadeia RAG M5.14–M5.24: índice de produção, contrato, adapter e context builder
01/10/2026	Integração do Mem0, pipeline documental, consolidação operacional
02/10/2026	Autenticação humana, RBAC, OCR e geração DOCX
09–10/10/2026	Stack de observabilidade (Prometheus, Grafana, node-exporter)
14/10/2026	Refinamento visual R1–R3, PKI/portas, Fundação C1/C2 (Cadastro Universal)
16/10/2026	C2-DOC-UX-R1 e R1.1: diagnóstico e correção dos vínculos documentais
19/10/2026	Verificação de estado que fundamenta este relatório

2.2 Infraestrutura

A VM Medusa está estável, com 9 dias e 10 horas de uptime no momento da coleta (boot em 09/10/2026 às 22:25).

```
Proxmox / pve-asus
        │
        ▼
VM Medusa (VMID 1001)
├── Ubuntu Server 26.04.1 LTS
├── 30 vCPU
├── 22 GiB utilizáveis (24 GiB configurados)
├── 735 GB de disco — 79 GB usados (12%)
├── RTX 5090 32 GB — passthrough
└── Docker + serviços de usuário (systemd --user)
```

- Disco: 619 GB livres.
- Memória: 2,2 GiB em uso, 20 GiB disponíveis.
- GPU: 1.102 MiB alocados, 0% de utilização no momento da coleta.
- Tailscale ativo, com 3 peers online.
- Não há indício de degradação; o swap registra 1,3 GiB usados, ponto de atenção leve para picos de OCR/indexação.

2.3 Containers

Nove containers, todos em execução, sem reinícios anormais:

Container	Imagem	Estado
`infra-postgres`	pgvector/pgvector:0.8.6-pg18-trixie	healthy
`infra-redis`	Redis	healthy
`infra-qdrant`	qdrant/qdrant:v1.19.1	running
`medusa-docling`	docling-serve-cu130:v1.34.0	healthy
`medusa-searxng`	SearXNG	healthy
`obs-prometheus`	prom/prometheus:v3.7.3	running
`obs-grafana`	grafana/grafana:12.3.1	running
`obs-node-exporter`	prom/node-exporter:v1.9.1	running
`u0137-ollama`	ollama/ollama	running

2.4 Serviços da aplicação

Quatro serviços de usuário, todos `enabled` e `active`, além de três timers:

```
davy-jones.service          → API + UI + administração     :8766
medusa-core.service         → recuperação de evidências     :8765
medusa-rag-worker.service   → ingestão documental persistente
medusa-caddy.service        → proxy reverso + TLS interno   :5080 / :3443 / :3000
```

- `medusa-davy-health.timer`: health check a cada 5 minutos.
- `medusa-davy-backup.timer`: backup operacional diário (madrugada).
- `medusa-remote-backup.timer`: replicação verificada para SMB via Tailscale.

2.5 Medusa Core e corpus

A recuperação de evidências está operacional, com geração desligada por decisão de projeto.

```
Medusa Core :8765
├── health ................ live=true, generation_enabled=false
├── corpus canônico ....... 2 documentos, 206 chunks
├── coleção ............... medusa_rag_chunks_v1
├── dimensão .............. 2560
├── distância ............. Cosine
└── pontos ................ 206, status green
```

O índice de produção permanece congelado em M5.14 (206 pontos, UUIDv5, payloads verificados). Nenhuma reconstrução ou mutação foi feita fora de etapa autorizada.

2.6 Workspace documental e RAG

O pipeline documental está ativo e separado do corpus canônico:

```
Upload → Docling → RAG Worker → chunking → embeddings → Qdrant
                                              │
                                              └── medusa_workspace_<hash>
```

- 3 documentos publicados no workspace.
- 5 revisões imutáveis preservadas.
- 879 pontos distribuídos em 5 coleções de workspace.
- Worker ativo desde 09/10, com 18 minutos de CPU acumulada.
- Modelo de embedding: `qwen3-embedding:4b`, dimensão 2560.
- `medusa_rag_chunks_v1` permanece inalterado (`canonical_unchanged=true`).
- Isolamento de memória confirmado (`memory_isolated=true`).

2.7 Qdrant

Doze coleções, todas `green`:

Coleção	Pontos	Dimensão
`medusa_rag_chunks_v1`	206	2560
`medusa_structured_docs_v1`	625	2560
`medusa_docs_v1`	30	2560
`hermes_eng_memory_v2`	92	2560
`davy_jones_mem0_v1`	3	2560
`mem0migrations`	0	2560
`medusa_recovery_*`	15	2560
5 coleções `medusa_workspace_*`	879	2560
Total	**1.850**	—

As coleções protegidas (`hermes_eng_memory_v2` , `mem0migrations` , `medusa_docs_v1` , `medusa_structured_docs_v1` , `medusa_rag_chunks_v1`) permanecem sob a regra de não mutação sem autorização explícita.

2.8 Ollama e GPU

Seis modelos disponíveis, todos locais:

```
RTX 5090 32 GB
      │
      ▼
Ollama :11434
├── qwen3:8b .................. 5,2 GB
├── qwen3-embedding:4b ........ 2,5 GB
├── gemma4-64k ................ 19,9 GB
├── bge-m3 .................... 1,2 GB
├── snowflake-arctic-embed2 ... 1,2 GB
└── embeddinggemma ............ 0,6 GB
```

`qwen3:8b` atende o Agent e o Mem0; `qwen3-embedding:4b` atende o RAG. A GPU responde e não há contenção no momento.

2.9 Mem0

A memória operacional está integrada e isolada do corpus jurídico:

```
Mem0
├── coleção ......... davy_jones_mem0_v1
├── pontos .......... 3
├── dimensão ........ 2560
├── embedding ....... qwen3-embedding:4b
├── LLM ............. qwen3:8b
├── telemetria ...... false
└── escopo .......... local_only
```

A separação `Mem0 ≠ corpus jurídico` continua respeitada. As 23 buscas de memória e 1 gravação registradas no histórico de jobs confirmam uso real, não apenas instalação.

2.10 SearXNG e pesquisa Web

- `medusa-searxng` saudável, exposto apenas em `127.0.0.1:8081`.
- 6 jobs `web_search` concluídos.
- Integração via `web_search` do Agent, mantendo separação entre informação externa e evidência interna.

2.11 OCR documental

- 19 jobs `document_ocr` concluídos.
- 33 revisões `ocr_review` concluídas.
- Endpoint de imagem servido por job e imagem (`/documents/ocr/<job>/files/<image>`).
- Revisão humana obrigatória preservada; o OCR não cria pessoas nem vínculos automaticamente.
- Contratos, hashes e arquivos originais intactos.

2.12 Geração DOCX controlada

- 4 jobs `document_docx` concluídos.
- 4 revisões `docx_review` concluídas.
- Seleção amigável de documentos com validação obrigatória (`validate_selection`).
- Templates e contrato de geração preservados.
- Nenhuma geração ocorre sem confirmação e revisão humanas.

2.13 Cadastro Universal (C1/C2)

Sete tabelas relacionais no SQLite, com migração aditiva e idempotente:

```
registry_persons
registry_identifiers
registry_roles
registry_person_history
registry_doc_labels
registry_doc_label_history
registry_doc_person_links
```

Funcionalidades entregues:

- Pessoa PF/PJ, com validação de CPF/CNPJ quando informado.
- Busca por nome, tipo e papel.
- Edição, status e histórico auditável.
- Identificadores técnicos separados dos nomes amigáveis.
- Rótulo documental sem renomear arquivo, hash ou `document_id`.
- Vínculo documento–pessoa com estado `CONFIRMED`, `PENDING` ou `DISPUTED`.
- Confirmação humana explícita obrigatória.
- Nenhuma criação automática de pessoa a partir do OCR.
- Nenhuma fusão de cadastros por igualdade de nome.

Estado atual: 3 pessoas (dados de teste do administrador), 0 rótulos, 0 vínculos. O fluxo foi validado por API e por reprodução isolada; a validação no navegador autenticado segue pendente de senha humana.

2.14 Biblioteca de documentos e correção R1.1

O erro `invalid_state` foi diagnosticado e corrigido em 16/10/2026.

Causa-raiz: os `<option>` de estado eram criados sem atributo `value`. O navegador enviava o texto visível (`"Confirmado"` / `"Pendente"`), mas o contrato aceita apenas `CONFIRMED`, `PENDING` e `DISPUTED`. O backend rejeitava antes de qualquer `INSERT`.

Correção: apenas `app/registry_views.py`, com valores contratuais explícitos, cinco estados visuais distintos (encontrada → selecionada → pendente → salvando → salva/recusada) e mensagens em português. O backend e as regras de estado não foram alterados.

Validação: 23/23 testes unitários, 10/10 testes de persistência, 9/9 de segurança. O vínculo persiste, aparece no histórico e é recuperado após releitura.

2.15 Autenticação e RBAC

- Login humano com sessão, cookie e CSRF.
- Dois usuários: 1 ADMIN e 1 USER, ambos habilitados e não bloqueados.
- Página e APIs administrativas restritas a ADMIN; USER recebe 403; sem sessão, 401.
- Rate limit de login ativo (1 evento registrado nos últimos 7 dias).
- Auditoria de autenticação: 15 logins com sucesso, 13 falhas, 11 logouts e 1 revogação de sessão na última semana.

- Sessões não são criadas a partir do Cadastro Universal.

2.16 Interface

A UI própria do Davy-Jones é a interface real do projeto — não a Open WebUI do desenho original.

Ela possui oito visões principais (home, evidência, pesquisa, resposta, ferramentas, memória, configuração, diagnóstico) e o grupo Administração, com Cadastro Universal e Biblioteca de documentos.

Entregas visuais concluídas:

- R1–R3: cartões de status em grade, título à esquerda, selo à direita, descrição abaixo, responsividade.
- Login e RBAC integrados ao menu.
- Cadastro e Biblioteca integrados ao menu administrativo, ocultos para USER.
- Nenhum uso de `innerHTML` nos pontos sensíveis; conteúdo dinâmico via `textContent`.

2.17 Caddy, TLS e exposição

O desenho original previa Nginx/HTTPS :443. A implementação real usa Caddy com CA interna:

```
https://medusa.escritorio.local:5080   → Davy-Jones :8766
https://medusa.escritorio.local:3443   → Davy-Jones :8766
http://medusa.escritorio.local:3000    → Davy-Jones :8766
```

- Bind restrito a `192.168.15.250`.
- Acesso externo à sub-rede negado (`@outside abort`).
- Cabeçalhos `X-Content-Type-Options`, `X-Frame-Options` e `Referrer-Policy`.
- Proxy injeta segredo interno e remove `Authorization` do cliente.
- Certificado interno válido, com renovação automática (janela de 12 horas).
- Grafana e Prometheus escutam apenas em loopback.
- Tailscale expõe o Davy-Jones em `100.126.89.55:8766` para administração remota.

2.18 Backup e continuidade

```
Medusa
  ↓
backup operacional diário (SQLite + código + workspace + Qdrant)
  ↓
/home/tlandskron/medusa/backups/operational-<timestamp>/
  ↓
SMB \\100.94.111.91\backups\Medusa  (Tailscale)
```

- 31 pontos de backup, 2,9 GB no total.
- Último backup: 19/10/2026, 121 MB, estado `COMPLETE`.
- Manifesto com SHA-256 por arquivo.

- Replicação remota verificada: 5 backups recentes `ALREADY_VERIFIED`, estado final `COMPLETE`.
- Snapshot SQLite feito por API de backup (WAL-safe), não por cópia bruta.

2.19 Observabilidade — base instalada

Componente	Versão	Estado
Prometheus	3.7.3	🟢
Grafana	12.3.1	🟢
node-exporter	1.9.1	🟢

- Prometheus com retenção de 15 dias / 8 GB.
- Dois targets ativos e `UP`: `node` e `prometheus`.
- Dashboard "Medusa — Overview Host" com 11 painéis (uptime, CPU, núcleos, RAM, disco, load, rede, processos).
- Grafana com datasource Prometheus provisionado e login obrigatório.
- Acesso apenas por túnel SSH em `127.0.0.1:3000`.

2.20 Documentação e governança

- Estado oficial do RAG: M5.23 congelado, com SHA-256 registrado.
- Último índice de produção: M5.14, 206 pontos.
- M5.24 executada, mas `REVIEW_REQUIRED` e não congelada.
- M5.24-D1/D2/D3-R1 concluídos como diagnósticos.
- M5.24-D4 autorizada e construída, **não executada**.
- M5.25+ permanece `NOT_AUTHORIZED`.
- Regras de não mutação de Qdrant, não criação de scripts sem autorização e não rollback implícito preservadas.

2.21 Testes

Execução atual com o interpretador da aplicação (`.venv-mem0`, Python 3.11):

Módulo	Testes	Resultado
`test_davy.py`	4	OK
`test_grounding.py`	20	OK
`test_human_auth.py`	26	OK
`test_legal_facts.py`	23	OK
`test_legal_planner.py`	29	OK
`test_memory.py`	4	OK

Módulo	Testes	Resultado
`test_rag_api.py`	3	OK
`test_registry.py`	23	OK
`test_ui_access.py`	2	OK
Demais módulos (DOCX, OCR, review, tools, web)	0 executáveis isoladamente	OK (dependem de fixtures/ambiente)
Total	**134**	**0 falhas**

Além disso, existem 35 scripts de verificação viva (UI, OCR, DOCX, RBAC, RAG, grounding, planner, tools) disponíveis para reexecução.

---

3. O que está em curso (configuração e validação)

3.1 Observabilidade completa

A base está no ar, mas a cobertura ainda é de **host**, não de aplicação.

Já feito:

- Métricas de CPU, memória, disco, rede, load e processos.
- Dashboard provisionado.
- Retenção definida.

Ainda em configuração:

- Métricas por container (cAdvisor foi descartado por incompatibilidade; o caminho alternativo ainda não foi escolhido).
- Métricas de negócio da aplicação (jobs, fila, latência de RAG, OCR, erros HTTP).
- Agregação de logs (Loki + Promtail ou equivalente).
- Alertas (Alertmanager) e disponibilidade externa (Uptime Kuma).
- Exposição do Grafana: a configuração declara `https://grafana.escritorio.local:6001`, mas a porta 6001 não está publicada; hoje o acesso é só por túnel SSH em `127.0.0.1:3000`.

3.2 Geração jurídica controlada (M5.24)

O pipeline de recuperação está congelado; a geração ainda não está liberada.

```
M5.14  índice de produção congelado ........ 206 pontos
M5.20  especificação consolidada ........... congelada
M5.21  contrato de retrieval ............... congelado
M5.22  adapter ............................. congelado
M5.23  context builder ..................... congelado
M5.24  primeira geração controlada ......... REVIEW_REQUIRED
M5.24-R1 certificação determinística ....... REVIEW_REQUIRED_NOT_FROZEN
M5.24-D4 não determinismo de embeddings .... AUTORIZADA, NÃO EXECUTADA
M5.25+ ..................................... NOT_AUTHORIZED
```

O defeito central é conhecido: duas chamadas de embedding para a mesma pergunta produziram vetores diferentes (`max_abs_delta=0.0066` em um dos casos), embora a busca no Qdrant com o mesmo vetor seja reprodutível. Isso afeta reprodutibilidade de ranking e certificação literal de citações — não a integridade do índice.

Consequência prática: `generation_enabled=false`. O sistema recupera e apresenta evidências; não gera resposta jurídica autônoma.

3.3 Validação do Cadastro Universal em navegador

O fluxo foi corrigido e testado por API, mas a validação ponta a ponta no navegador autenticado depende de senha humana.

O que falta:

- Abrir `/admin/registry` com sessão ADMIN real.
- Percorrer: localizar documento → visualizar original → pesquisar pessoa → selecionar → confirmar → salvar → recarregar → conferir histórico.
- Confirmar visualmente os cinco estados da associação.
- Repetir em janela estreita.

Não é permitido simular essa etapa como aprovada.

3.4 Rótulo documental removido

Durante os testes vivos da R1.1, um script de validação renomeou e depois removeu o rótulo de um documento OCR real do administrador. Pessoas, jobs, revisões, arquivos e vínculos (inexistentes) permaneceram intactos.

Estado atual: `registry_doc_labels = 0` e `registry_doc_label_history = 0`.

A reconstrução foi proposta com documento identificado e texto conhecido, mas **não foi executada** por exigir autorização. O histórico original (data e autor) não é recuperável; a reconstrução seria registrada como tal.

3.5 Camada de dados

Existem três mecanismos, com papéis distintos:

Mecanismo	Uso atual	Situação
SQLite `jobs.sqlite3`	fila, jobs, autenticação, Cadastro Universal, histórico	🟢 fonte da verdade
Qdrant	vetores do RAG, workspace, Mem0	🟢 em uso
PostgreSQL + pgvector	databases `medusa` e `davy_jones` criados, extensão `vector` disponível, sem tabelas de aplicação	🟡 preparado, ocioso
Redis	container saudável, com autenticação, sem uso direto identificado no fluxo principal	🟡 disponível, ocioso

Não há defeito nisso: o SQLite foi escolhido por consistência e backup em arquivo único. Mas a arquitetura precisa decidir, antes de crescer, se PostgreSQL/pgvector será adotado ou removido do caminho crítico.

3.6 Acesso externo

O desenho original previa `Usuário → Nginx → HTTPS :443`. O estado real é:

```
LAN 192.168.15.0/24
        |
        ▼
Caddy :5080 / :3443 (TLS interno)
        |
        ▼
Davy-Jones :8766 (loopback)

Remoto:
Tailscale → 100.126.89.55:8766
```

Isso é funcionalmente equivalente e mais simples, mas não é o desenho original. A decisão pendente é: manter Caddy + Tailscale como padrão, ou preparar exposição :443 com Nginx/Caddy e certificado público.

3.7 Open WebUI

A Open WebUI não está instalada (nenhum container, binário ou pacote). A UI própria já cobre consulta, agente, OCR, DOCX, memória, ferramentas, diagnóstico e administração.

A decisão pendente não é técnica, é de arquitetura: manter a UI própria como interface definitiva ou reintroduzir a Open WebUI como camada unificada. Enquanto a decisão não for tomada, não se deve instalar nem integrar nada.

3.8 Backup e retenção

O backup local e a réplica SMB estão funcionando. O que ainda está em definição:

- Política formal de retenção (quantos dias/meses, o que expurgar).
- Cópia independente em nuvem (o destino atual é um Windows 11 via Tailscale).

- Teste de restauração integral documentado e periódico.
- Confirmação de criptografia/ACL no destino remoto, porque o backup contém dados pessoais do Cadastro Universal.

---

4. O que falta concluir

4.1 Prioridade alta — desbloqueiam outras etapas

#	Item	Por que importa	Critério de conclusão
P1	Executar M5.24-D4	Único diagnóstico autorizado que pode explicar o não determinismo dos embeddings	D4 executada, evidências LOG/TXT/JSON, conclusão registrada sem ampliar escopo
P2	Validar o fluxo documental no navegador	A correção R1.1 está provada por API, não por uso real	Sessão ADMIN real, vínculo salvo, reload, histórico e cinco estados conferidos
P3	Reconstruir (ou dispensar formalmente) o rótulo removido	Hoje há 0 rótulos e 0 vínculos em produção	Autorização explícita + reconstrução registrada como tal, ou decisão de não reconstruir

4.2 Prioridade média — operação e continuidade

#	Item	Situação atual	Próximo passo
P4	Observabilidade completa	Só host; sem logs, alertas ou uptime	Escolher stack de logs, definir alertas, publicar Grafana com segurança
P5	Métricas de aplicação e containers	Ausentes	Definir exporter/coletor compatível e instrumentar jobs/fila/RAG
P6	Backup 3-2-1 e retenção	Local + SMB; sem nuvem nem política formal	Definir retenção, destino independente e teste de restauração
P7	Decisão sobre PostgreSQL/pgvector e Redis	Preparados, ociosos	Adotar no caminho crítico ou documentar como reserva
P8	Decisão sobre Open WebUI	Ausente	Manter UI própria ou planejar integração — sem instalação até decidir

4.3 Prioridade estrutural — evolução do produto

#	Item	Dependência
P9	Liberar geração jurídica controlada (M5.25+)	P1 e revisão da M5.24-R1
P10	Planner avançado do Agent	Decisão de arquitetura e autorização
P11	Consolidação de redes/volumes Docker e NAS	Planejamento de armazenamento

#	Item	Dependência
P12	Exposição externa :443 (se adotada)	Decisão P8/P7 e revisão de segurança

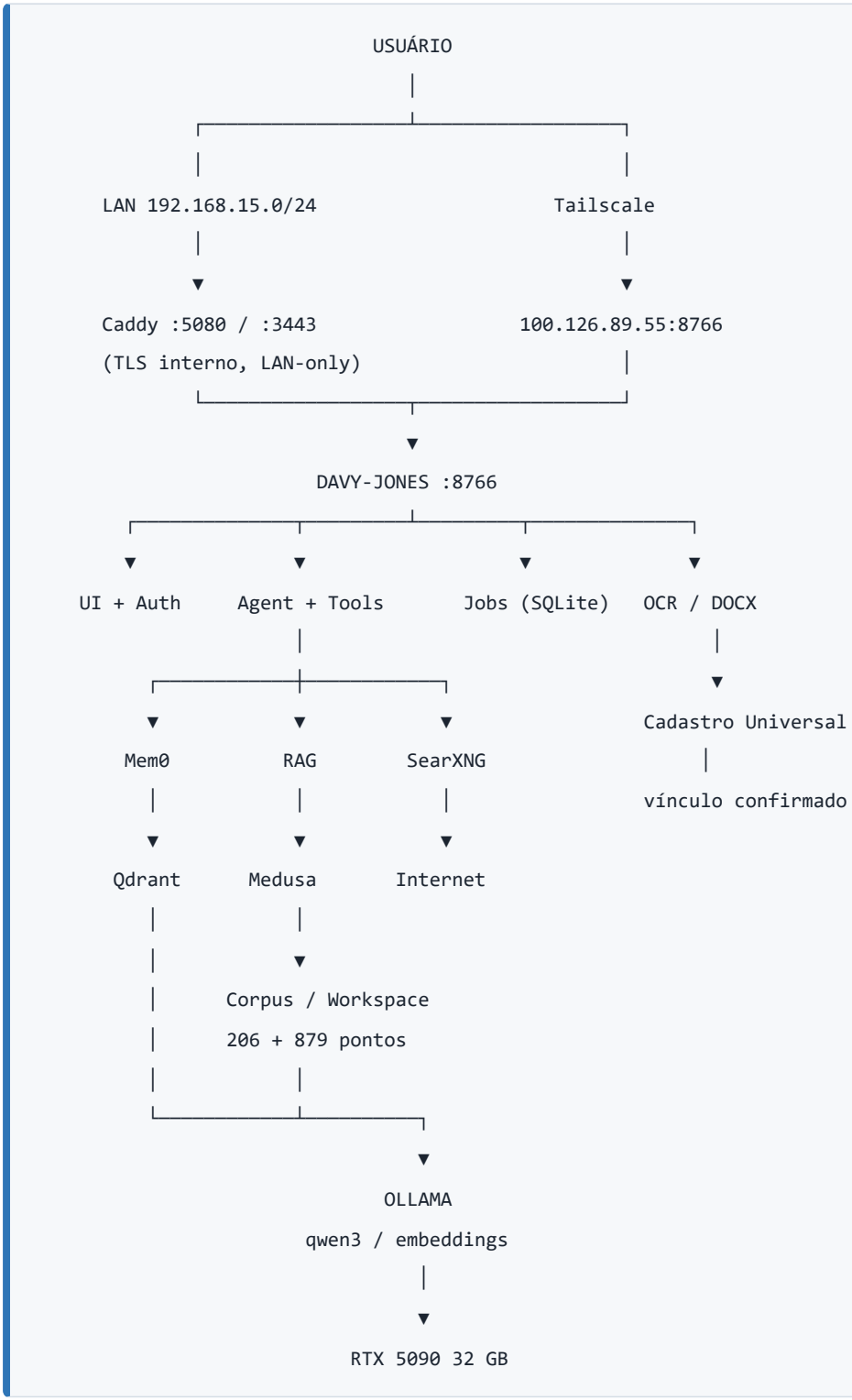
4.4 Sincronização documental

O documento de estado oficial mais recente no diretório é de **25/09/2026**; a transferência RAG é de **28/09/2026**. As etapas de outubro (auth, OCR/DOCX, UI R1–R3, C1/C2, R1.1 e observabilidade) não estão refletidas nele.

Isso não é falha técnica, mas é risco de governança: uma próxima conversa pode interpretar o estado como anterior ao que realmente existe. Falta um novo checkpoint oficial que consolide outubro.

---

5. Arquitetura atual



Leitura do diagrama: a UI e a autenticação vivem no Davy-Jones; o Agent acessa Mem0, RAG e SearXNG; OCR e DOCX alimentam o Cadastro Universal; o RAG usa o corpus/workspace e o Ollama; infraestrutura, observabilidade e backup são camadas de apoio, não caminhos de negócio.

```
INFRAESTRUTURA                    OBSERVABILIDADE
├── PostgreSQL + pgvector 🟡      ├── Prometheus 🟢
├── Redis 🟡                      ├── Grafana 🟢
├── Docling 🟢                    ├── node-exporter 🟢
├── Qdrant 🟢                     ├── Loki 🔴
└── Docker 🟢                     ├── Alertmanager 🔴
                                  └── Uptime Kuma 🔴

BACKUP
├── local diário 🟢
├── SMB/Tailscale verificado 🟢
└── nuvem independente 🔴
```

---

6. Evidências da verificação

Verificação	Resultado
`systemctl --user is-active` dos 4 serviços	todos `active`
`systemctl --user --failed`	0 falhas
Containers em execução	9 de 9
`/health` Medusa Core (:8765)	200, `live=true`
`/health` Davy-Jones (:8766)	200, `worker_alive=true`
`/ready`	`ready=true`
`/rag/status`	`ready=true`, `worker_alive=true`
`/memory/status`	`ready=true`, 3 pontos
Qdrant `/healthz`	200
Docling `/health`	200
SearXNG	200
Login HTTPS :5080	200
Login HTTPS :3443	200
Login HTTP :3000	200
Prometheus targets	2/2 `UP`
Grafana `/api/health`	`database=ok`
Backup mais recente	`COMPLETE`, 121 MB

Verificação	Resultado
Réplica SMB	`ALREADY_VERIFIED` nos 5 últimos
Testes unitários	134/134, 0 falhas
Disco	12% usado, 619 GB livres
GPU	RTX 5090, 1.102 MiB, 0%

---

7. Riscos e pontos de atenção

1. **Estado oficial desatualizado.** O checkpoint formal é de 25/09; houve pelo menos seis etapas relevantes depois disso. Risco de retrabalho ou de decisão baseada em estado antigo.
2. **Zero vínculos documentais em produção.** A funcionalidade está implementada e testada, mas ainda não foi usada em um caso real. O primeiro uso deve ser acompanhado.
3. **Rótulo removido.** Há 0 rótulos e 0 históricos de rótulo. A reconstrução exige autorização; não deve ser feita em silêncio.
4. **Não determinismo de embeddings.** Afeta reprodutibilidade e certificação literal. Não afeta a integridade do índice nem autoriza reindexação.
5. **Observabilidade sem alertas.** Se um serviço cair às 3h, ninguém é notificado. O health check grava JSON, mas não alerta.
6. **Backup com dados pessoais.** O backup contém o SQLite do Cadastro Universal. É preciso confirmar criptografia e ACL no destino SMB e em qualquer cópia futura.
7. **Dependência do host Windows para a réplica remota.** Se o Windows 11 ou o Tailscale estiverem offline, a réplica para. O backup local continua.
8. **VM única, sem HA.** Não há redundância de host. A continuidade depende de backup e de tempo de reconstrução.
9. **Swap em uso.** 1,3 GiB de swap com 20 GiB disponíveis. Não é crítico, mas merece monitoramento durante OCR/indexação pesada.
10. **Certificado interno de 12 horas.** A renovação é automática; se o Caddy falhar, a janela de validade é curta. O monitoramento do Caddy é obrigatório.
11. **PostgreSQL/Redis ociosos.** Não é erro, mas consome memória e aumenta a superfície de manutenção sem entregar valor enquanto não houver decisão.
12. **Grafana anunciado em :6001, mas não publicado.** A configuração e a realidade divergem; hoje só há túnel SSH.

---

8. Sequência recomendada

A ordem abaixo respeita as dependências e não autoriza execução por si só.

```
1. Autorizar e executar D4
        ↓
2. Revisar M5.24-R1 com a evidência da D4
        ↓
3. Validar o fluxo documental no navegador (sessão ADMIN real)
        ↓
4. Decidir sobre o rótulo removido
        ↓
5. Completar observabilidade (logs + alertas + uptime)
        ↓
6. Fechar backup 3-2-1 + retenção + teste de restauração
        ↓
7. Decidir camada de dados (SQLite vs PostgreSQL/pgvector)
        ↓
8. Decidir Open WebUI e exposição externa
        ↓
9. Só então planejar M5.25+ e o planner avançado
```

Não inverter os passos 1–2 com os passos 5–8. A geração jurídica não deve ser liberada antes de a reprodutibilidade estar caracterizada; e a observabilidade não deve ser expandida antes de a decisão de interface e de camada de dados estar tomada, para não instrumentar duas vezes.

---

9. O ponto mais importante

O projeto não está em fase de reconstrução. Está em fase de fechamento.

O núcleo que realmente importa já funciona:

> **Medusa Core + Qdrant + corpus de 206 chunks + workspace + Docling + RAG Worker + Ollama/RTX 5090 + Mem0 + SearXNG + Davy-Jones + UI + Auth/RBAC + OCR + DOCX + Cadastro Universal + backup local e remoto.**

O que resta são **camadas de produção e decisões de arquitetura**:

1. caracterizar o não determinismo dos embeddings (D4) e revisar a geração;
2. validar o fluxo documental em uso real;
3. completar logs, alertas e disponibilidade;
4. fechar a continuidade externa e a retenção;
5. decidir PostgreSQL/pgvector, Open WebUI e exposição :443;

6. atualizar o checkpoint oficial do projeto.

A diferença entre o relatório de 01/10 e este é grande: em 01/10, Mem0, RAG, Docling, SearXNG, ferramentas, UI administrativa e observabilidade estavam pendentes. Hoje todos esses blocos existem e estão no ar. O que mudou não foi o discurso — foi a verificação.

Conclusão executiva: o Caribbean Pirates GPT possui hoje um núcleo jurídico-documental operacional, autenticado, auditável e com backup verificado. O caminho crítico restante é curto e conhecido: D4 → revisão da geração → validação em navegador → observabilidade/continuidade → decisões estruturais. Nenhuma dessas etapas exige desmontar o que já está funcionando.

Fim do relatório — 19/10/2026.